3. What are Software Myths? Explain any four common software myths with suitable examples.

Answer :

Software Myths are false beliefs or wrong ideas about software and software development. These myths can lead to poor planning, wrong decisions and project failure. Understanding and avoiding them helps in building better software.

Four Common Software Myths :

① Myth : If a project is behind schedule, add more people.
Adding more people to a delayed project will only increase communication and coordination problems.
Example : A late project team gets more members, but they take time to understand the work, so the delay increases.

② Myth : Once the software is working, the job is done.
Software needs regular maintenance to fix bugs, improve performance and adapt to new needs.
Example : A banking app works well at first, but after some time it needs updates for security and new features.

③ Myth : Good programmers can write perfect code.
Even the best programmers can make mistakes. Every code needs testing and review.
Example : A small coding error by an expert may crash the system if not tested.

④ Myth : Software development is just coding.
Coding is only a small part. Planning, requirements, design, testing and maintenance are equally important.
Example : Without proper design and testing, even well-written code can fail in the real world.

4. Explain the Phased Development Life Cycle and describe its major phases.

Answer :

Phased Development Life Cycle is a step-by-step approach to build software. The project is divided into phases. Each phase has specific activities and deliverables. One phase is completed before the next one starts.

Major Phases :

① Requirements Phase – Understand what the users need.
- Collect, analyze and document requirements.

② Design Phase – Plan how the software will be built.
- Create system design and detailed design.

③ Implementation Phase (Coding) – Write the code.
- Develop the software as per the design.

④ Testing Phase – Check the software for errors.
- Test and fix defects to ensure it works correctly.

⑤ Deployment Phase – Deliver the software to users.
- Install and make the software available to users.

⑥ Maintenance Phase – Support and improve the software.
- Fix bugs, improve performance and add new features.

These phases help in building quality software in an organized and controlled way.

6. Explain the Spiral Model and describe how it helps in managing risks in software development.

Answer :

The Spiral Model is a risk-driven model of software development. It combines the features of iterative development and waterfall model. The project is divided into small cycles called spirals. Each spiral represents a phase of the project. With every cycle, the product becomes more complete and better.

Phases of Spiral Model :

Each spiral has four main activities :

① Planning : Identify objectives, alternatives and constraints. Estimate cost and schedule.

② Risk Analysis : Identify possible risks. Evaluate and reduce these risks.

③ Engineering : Develop and test the product for that cycle. Create prototypes or models if needed.

④ Evaluation : Review the results with the customer and plan the next cycle.

How it helps in managing risks :

- Risk analysis is done in every cycle. So, problems are found early.
- High risks are handled in the initial cycles, when changes are easier and cheaper.
- By using prototypes, uncertain requirements can be tested and clarified.
- Continuous feedback from users reduces the chance of failure.
- It allows flexible changes based on risk and customer needs.

Thus, the Spiral Model helps in identifying, evaluating and reducing risks throughout the development process.

7. Compare the Waterfall Model and Evolutionary Model with respect to development approach, requirements, and customer involvement.

Answer :

Waterfall Model is a linear and sequential approach, while Evolutionary Model is an iterative approach. They differ in their development approach, handling of requirements and customer involvement as given below.

Aspect	Waterfall Model	Evolutionary Model
1. Development Approach	• Linear and sequential. • Each phase is completed before the next one starts.	• Iterative and incremental. • System is built in small parts and improved in each iteration.
2. Requirements	• Requirements are gathered completely at the beginning. • Changes in requirements are difficult and costly to handle.	• Requirements are collected partially and can change during development. • Easy to accommodate changes.
3. Customer Involvement	• Customer is involved mainly at the beginning (for requirements) and at the end (for final product). • Less customer interaction in between.	• Customer is involved throughout the process. • Regular feedback is taken and changes are done as per customer needs.

In short, Waterfall works best when requirements are clear and stable, while Evolutionary works best when requirements are not fully known or are changing.

8. Explain the major activities involved in Software Project Planning and Estimation.

Answer :

Software Project Planning is the first and very important step in any software project. It helps us decide how the project will be done, how much time it will take, how much it will cost and what resources are needed. Estimation is the process of finding the effort, time and cost required for the project.

The major activities involved are :

① Define Project Scope
- Understand the problem and customer needs.
- Decide what is to be included and what is not.

② Break Down the Work
- Divide the whole project into smaller tasks.
- Create a Work Breakdown Structure (WBS).

③ Estimate Effort
- Estimate the effort needed for each task.
- Use methods like expert judgment, analogy or historical data.

④ Estimate Time (Schedule)
- Find the time required to complete each task.
- Prepare the project schedule and set milestones.

⑤ Estimate Cost
- Calculate the cost based on effort, time and resources.
- Include hardware, software, tools and training cost if needed.

⑥ Resource Planning
- Identify the people, skills, tools and equipment required.
- Assign resources to tasks.

⑦ Risk Planning
- Identify possible risks (e.g., requirement change, delay, lack of staff).
- Plan actions to avoid or reduce the impact of risks.

⑧ Prepare Project Plan
- Combine all the above activities into a complete project plan.
- Plan includes scope, schedule, cost, resources, risks and quality plan.

These activities help in using time, cost and resources properly and make the project more successful.

9. Explain the process of Gathering Requirements and discuss its importance in software development.

Answer :

Gathering requirements means collecting complete and accurate information about what the users need from the software. It is the first and most important step in software development.

Process of Gathering Requirements :

① **Identify stakeholders** : Find out who will use the software and who is interested in it.

② **Elicit requirements** : Collect information using methods like interviews, questionnaires, observation and meetings.

③ **Analyze requirements** : Study and understand the collected information. Remove unclear or conflicting points.

④ **Document requirements** : Write the requirements in a clear and organized way in a document called SRS (Software Requirements Specification).

⑤ **Validate requirements** : Review the requirements with users to make sure they are correct and complete.

⑥ **Manage requirements** : Keep track of changes in requirements and update the document if needed.

Importance of Gathering Requirements :

- Helps in understanding exactly what the users need.
- Reduces changes and problems during development.
- Saves time and cost.
- Provides a clear plan for design, coding and testing.
- Improves customer satisfaction.
- Leads to successful delivery of the software.

So, proper requirement gathering is essential for building the right software that solves the real problem of users.